

ECE1778: Creative Applications for Mobile Devices

Chen Ying

Assistant Professor, Teaching Stream

Department of Electrical and Computer Engineering

University of Toronto

Last Week's Lecture

Styling and UI Design in React Native

- Styling with [StyleSheet](#) and Flexbox
- New core components: [Pressable](#), [Image](#), [Modal](#)
- Card-like layouts
- Reusable components & styles



```
project-root/  
├── components/  
│   └── PrimaryButton.tsx  
├── constants/  
│   └── colors.ts  
└── styles/  
    └── globalStyles.ts
```

Styling Basics

```
<View style={styles.container} />
<View style={[styles.base, styles.override]} />
```

React Native uses **JavaScript objects** for styling

- Apply styles via `style` prop
- `style` prop accepts one object or an array of objects (merged in order)

Responsive Design

Why is it important?

- Support diverse devices: Phones, tablets
- Handle multiple orientations: Portrait, landscape
- Enhances user experience across screen sizes

Goal: Create layouts that adapt seamlessly to device dimensions

Example Review

App.tsx

```
1 import { Pressable, Image, Alert, StyleSheet } from "react-native";
2
3 export default function App() {
4   return (
5     <Pressable
6       style={styles.container}
7       onPress={() => Alert.alert("Image pressed!")}
8     >
9       <Image
10        style={styles.fullScreenImage}
11        source={require("./assets/images/night-sky.jpg")}
12      />
13    </Pressable>
14  );
15 }
16
17 const styles = StyleSheet.create({
18   container: {
19     flex: 1,
```

`width: "100%"` and `height: "100%"` make the image fill the container

Example Review

```
fullScreenImage: {  
  width: "100%",  
  height: "100%",  
  resizeMode: "cover", // Ensures image scales properly  
},
```

.

Responsive Design

React Native provides APIs to handle dynamic layouts:

- `Dimensions.get("window")`
- Retrieve static screen width and height
- `useWindowDimensions` hook
- Provide live, reactive dimensions that update on resize or rotation

Dimensions.get

`Dimensions.get("window")`: Return screen width & height

- Doc: reactnative.dev/docs/dimensions

Example:

```
import { Dimensions } from "react-native";

const deviceWidth = Dimensions.get("window").width;
const deviceHeight = Dimensions.get("window").height;

console.log(deviceWidth, deviceHeight);
```


Dimensions.get: Example

Create a skyblue box sized at 80% of screen width and 20% of screen height

App.tsx

```
1 import { View, Dimensions, StyleSheet } from "react-native";
2
3 export default function App() {
4   const { width: deviceWidth, height: deviceHeight } = Dimensions.get("wi
5
6   return (
7     <View style={styles.container}>
8       <View
9         style={{
10           width: deviceWidth * 0.8,
11           height: deviceHeight * 0.2,
12           backgroundColor: "skyblue",
13         }}
14       />
15     </View>
16   );
17 }
18
19 const styles = StyleSheet.create({
```

Static: Not update automatically on rotation

useWindowDimensions

Provide **live** width & height

- Update automatically on rotation or screen resize
- Doc: reactnative.dev/docs/usewindowdimensions

```
1 import { View, useWindowDimensions, StyleSheet } from "react-native";
2
3 export default function App() {
4   const { width: deviceWidth, height: deviceHeight } = useWindowDimension
5
6   return (
7     <View style={styles.container}>
8       <View
9         style={{
10           width: deviceWidth * 0.8,
11           height: deviceHeight * 0.2,
12           backgroundColor: "skyblue",
13         }}
14       />
15     </View>
16   );
17 }
18
19 const styles = StyleSheet.create({
```

Configure `app.json`

Enable rotation support in `app.json`

- Update the `orientation` key in `app.json` to explicitly allow rotation

```
{
  "expo": {
    "orientation": "default" // Allows both portrait and landscape
  }
}
```

- `"default"`: Permits both orientations
- Use `"portrait"` or `"landscape"` to lock orientation if needed

Conditional Sizing

Dynamically adjust UI based on device screen size

Example: A sky blue box with adaptive width

- 150 dp for compact phones (<380 dp, e.g., iPhone SE)
- 300 dp for standard phones/tablets (≥380 dp, e.g., iPhone 16, Galaxy S25)

```
<View
  style={{
    width: deviceWidth < 380 ? 150 : 300, // Conditional sizing
    height: deviceHeight * 0.2,
    backgroundColor: "skyblue",
  }}
/>
```

Units in React Native

- **px (pixel)**: Actual dot on the physical screen
 - Size depends on device density (dpi, dots per inch)
 - Not consistent across devices
- **dp (density-independent pixel)**: React Native's default unit for styling
 - 1dp = 1px on a 160 dpi screen
 - Scales automatically on higher-density screens
 - Keep UI about the same physical size

dp vs. px

Example: `width: 300` → 300 dp

- 160 dpi screen → 300 px
- 320 dpi (2x density) → 600 px
- 480 dpi (3x density) → 900 px

Recap: Responsive Design

- Use `Dimensions.get("window")` for static measurements
- Use `useWindowDimensions()` for dynamic, reactive layouts
- Apply conditional logic (e.g., `width < 380 ? 150 : 300`) for device-specific sizing
- Combine with Flexbox, percentages, and `app.json` orientation settings

Recap: Responsive Design

- **Tip:** Test on multiple devices/emulators to ensure adaptability
- **Advanced:** Combine with orientation events for custom behavior
- Detect rotation with [Dimensions.addListener\(\)](#) and adjust layouts, sizes, or features
- Example: iPhone Calculator shows scientific functions in landscape

Responsive Theming

Dark / Light Mode in React Native

- Users may prefer light or dark themes
- iOS and Android support system-wide appearance settings

Goal: Make your app adapt automatically to user preferences

Appearance API

Provide current color scheme of the device

```
import { Appearance } from "react-native";

const colorScheme = Appearance.getColorScheme();
console.log(colorScheme); // "light" or "dark"
```

- **Static:** Value does not update automatically if user changes system theme

useColorScheme Hook

React hook that returns current system theme

- **Reactive:** Automatically update on theme change

```
1 import { useColorScheme, View } from "react-native";
2
3 export default function App() {
4   const scheme = useColorScheme();
5   return (
6     <View style={{ backgroundColor: scheme === "dark" ? "#000" : "#fff" }}
7   );
8 }
```

With Constants/Styles

Centralize colors for easier dark/light theme support

```
constants/colors.ts
```

```
export const colors = {  
  light: { background: "#fff", text: "#000" },  
  dark: { background: "#000", text: "#fff" },  
};
```

Use in component

```
const scheme = useColorScheme();  
<View style={{ backgroundColor: colors[scheme].background }}>  
  <Text style={{ color: colors[scheme].text }}>Hello!</Text>  
</View>
```

Combine with `styles/globalStyles.ts` for reusable style patterns

Recap: Dark / Light Mode

- `Appearance.getColorScheme()`: Static value
- `useColorScheme()`: Reactive hook for theme changes
- For consistent UI in light & dark modes
- Centralize colors in `constants/colors.ts`
- Combine with Flexbox, `Pressable`, `Modal`, card, etc.

Advanced Styling Tools

- Animations
- **Animated API**: For granular and interactive control of specific values
- **LayoutAnimation API**: For animated global layout transactions
- **Accessibility APIs**: Build inclusive apps with accessible components
- UI Component Libraries with Integrated Styling
- Examples: **NativeUI**, **gluestack-ui**

Other Components for Styling & Layout

- [KeyboardAvoidingView](#): Automatically move content up when the keyboard appears
- [StatusBar](#): Control the appearance of the status bar (color, style, visibility)
- [ImageBackground](#): Display an image as a background and layer other components on top

Summary: Styling

- Styling with `StyleSheet` and Flexbox
- New core components: `Pressable`, `Image`, `Modal`
- Card-like layouts for structured content
- Reusable components & centralize styles/colors/fonts
- Responsive design and dark/light theming
- Advanced styling tools & other React Native components

Assignment 2: Style the Fitness Tracker app with consistent UI

Today's Lecture

Styling and UI Design in React Native

Navigation with React Navigation

Assignment 2: Navigation between three screens

- Home: Display activity list
- Add/Edit Activity: Log or update activities
- Activity Details: View individual activities

Navigation Basics

Core concepts for navigation in React Native mobile apps

- **Screen:** React component that defines the UI content displayed by a navigator
 - Example: `HomeScreen` renders app's home page
- **Navigator:** Component that manages screen transitions and user interactions
- **Routes:** Named destinations within a navigator, mapped to specific screens

React Navigation

Standard navigation library for React Native and Web apps

- Component-Based: Define navigation structure using React components
- Key Features:
 - Multiple navigators: Stack, Tab, Drawer
 - Type-safe with TypeScript
 - Cross-platform: iOS, Android, and Web
- Documentation: [**reactnavigation.org**](https://reactnavigation.org)

React Navigation: Setup

Documentation: Getting Started

Install core library

```
npm install @react-navigation/native
```

Install dependencies for an Expo-managed project

```
npx expo install react-native-screens react-native-safe-area-context
```

Navigators

Components that manage screen transitions and user interactions

- **Stack Navigator:** Manages a stack of screens where each new screen is pushed onto the top, with back navigation to previous screens
- **Tab Navigator:** Displays a tab bar for quick switching between multiple screens
- Variants: **Bottom Tabs**, **Material Top Tabs**
- Use Case: Parallel content categories (e.g., [Home](#), [Accounts](#), [Move Money](#) in a banking app)

Navigators

Manage screen transitions and user interactions

- **Stack Navigator:** Manages a stack of screens where each new screen is pushed onto the top, with back navigation to previous screens
- **Tab Navigator:** Displays a tab bar for quick switching between multiple screens
- **Drawer Navigator:** A slide-out menu for App-wide navigation
- Use Case: Settings, account management

Stack Navigator

Two types of stack navigators for different needs

- **Stack Navigator**: JavaScript-based, highly customizable navigation, full Web support
- **Native Stack Navigator**: Uses native APIs (UINavigationController on iOS, Fragment on Android) for native performance
- Mobile-focused, basic Web support, less customizable

Native Stack Navigator

Installation

```
npm install @react-navigation/native-stack
```


Navigator Configuration

Two ways to configure navigators

- **Static configuration:** Fixed at build time, cannot change at runtime
 - Write navigation configuration in an object
- **Dynamic configuration:** Flexible, can adapt at runtime
 - Write navigation configuration in React components (state/props)

Static Configuration

Key components for static setup

- `createNativeStackNavigator`: Creates a stack navigator from a configuration object
- `createStaticNavigation`: Wraps the navigator into a renderable component
- Called **once** at app startup
- Typically rendered at root of the app (`App.tsx`)

Static Configuration

- `createNativeStackNavigator`: Creates a stack navigator from a configuration object
- `createStaticNavigation`: Wraps the navigator into a renderable component

App.tsx

```
const RootStack = createNativeStackNavigator({
  screens: {
    Home: HomeScreen,
  },
});
const Navigation = createStaticNavigation(RootStack);

export default function App() {
  return <Navigation />;
}
```

Example

Define a stack navigator with two screens

App.tsx

```
1 import { createStaticNavigation } from "@react-navigation/native";
2 import { createNativeStackNavigator } from "@react-navigation/native-stac
3 import HomeScreen from "../screens/HomeScreen";
4 import DetailsScreen from "../screens/DetailsScreen";
5
6 const RootStack = createNativeStackNavigator({
7   screens: {
8     Home: HomeScreen,
9     Details: DetailsScreen,
10  },
11 });
12
13 const Navigation = createStaticNavigation(RootStack);
14
15 export default function App() {
16   return <Navigation />;
17 }
```

Project Structure



```
project-root/  
├── components/  
│   └── PrimaryButton.tsx  
├── constants/  
│   └── colors.ts  
├── screens/  
│   ├── HomeScreen.tsx  
│   └── DetailsScreen.tsx  
├── styles/  
│   └── globalStyles.ts  
├── App.tsx  
└── types.ts
```

HomeScreen

Basic screen component for the [Home](#) route

screens/HomeScreen.tsx

```
1 import { View, Text } from "react-native";
2 import { globalStyles } from "../styles/globalStyles";
3
4 export default function HomeScreen() {
5   return (
6     <View style={globalStyles.container}>
7       <Text style={globalStyles.headerText}>Home Screen</Text>
8     </View>
9   );
10 }
```

DetailsScreen

Basic screen component for the [Details](#) route

screens/DetailScreen.tsx

```
1 import { View, Text } from "react-native";
2 import { globalStyles } from "../styles/globalStyles";
3
4 export default function DetailsScreen() {
5   return (
6     <View style={globalStyles.container}>
7       <Text style={globalStyles.headerText}>Details Screen</Text>
8     </View>
9   );
10 }
```

Live Demo

App.tsx

```
1 import { createStaticNavigation } from "@react-navigation/native";
2 import { createNativeStackNavigator } from "@react-navigation/native-stac
3 import HomeScreen from "../screens/HomeScreen";
4 import DetailsScreen from "../screens/DetailsScreen";
5
6 const RootStack = createNativeStackNavigator({
7   screens: {
8     Home: HomeScreen,
9     Details: DetailsScreen,
10  },
11 });
12
13 const Navigation = createStaticNavigation(RootStack);
14
15 export default function App() {
16   return <Navigation />;
17 }
```


Static Configuration

All route configuration is specified as props to navigator

App.tsx

```
const RootStack = createNativeStackNavigator({
  screens: {
    Home: HomeScreen,
    Details: DetailsScreen,
  },
});
```

- **Home** route: Maps to **HomeScreen** component
- **Details** route: Maps to **DetailsScreen** component

Live Demo

App.tsx

```
1 import { createStaticNavigation } from "@react-navigation/native";
2 import { createNativeStackNavigator } from "@react-navigation/native-stac
3 import HomeScreen from "../screens/HomeScreen";
4 import DetailsScreen from "../screens/DetailsScreen";
5
6 const RootStack = createNativeStackNavigator({
7   screens: {
8     Home: HomeScreen,
9     Details: DetailsScreen,
10  },
11 });
12
13 const Navigation = createStaticNavigation(RootStack);
14
15 export default function App() {
16   return <Navigation />;
17 }
```

Static Configuration

`initialRouteName`: Initial route for the stack

App.tsx

```
const RootStack = createNativeStackNavigator({
  initialRouteName: "Home",
  screens: {
    Home: HomeScreen,
    Details: DetailsScreen,
  },
});
```

Screen Options

Customize individual screen appearance

App.tsx

```
1  const RootStack = createNativeStackNavigator({
2    initialRouteName: "Home",
3    screens: {
4      Home: {
5        screen: HomeScreen,
6        options: {
7          title: "Overview",
8        },
9      },
10     Details: DetailsScreen,
11   },
12 });
```

Screen Options

screenOption: Specify the same options for all screens in the navigator

App.tsx

```
1  const RootStack = createNativeStackNavigator({
2    initialRouteName: "Home",
3    screenOptions: {
4      headerStyle: { backgroundColor: colors.primary },
5      headerTintColor: colors.white,
6    },
7    screens: {
8      Home: {
9        screen: HomeScreen,
10       options: {
11         title: "Overview",
12       },
13     },
14     Details: DetailsScreen,
15   },
16 });
```

Move Between Screens

Add navigation functionality to [HomeScreen](#)

- Add a “Go to Details” button

screens/HomeScreen.tsx

```
1 import { View, Text, Button } from "react-native";
2 import { globalStyles } from "../styles/globalStyles";
3
4 export default function HomeScreen() {
5   return (
6     <View style={globalStyles.container}>
7       <Text style={globalStyles.headerText}>Home Screen</Text>
8       <Button
9         title="Go to Details"
10       />
11     </View>
12   );
13 }
```

Move Between Screens

useNavigation hook

screens/HomeScreen.tsx

```
1 import { View, Text, Button } from "react-native";
2 import { useNavigation } from "@react-navigation/native";
3 import { globalStyles } from "../styles/globalStyles";
4
5 export default function HomeScreen() {
6   const navigation = useNavigation();
7
8   return (
9     <View style={globalStyles.container}>
10       <Text style={globalStyles.headerText}>Home Screen</Text>
11       <Button
12         title="Go to Details"
13         onPress={() => navigation.navigate("Details")}
14       />
15     </View>
16   );
17 }
```

TypeScript Error

```
onPress={() => navigation.navigate("Details")}
```

- Error message: Argument of type 'string' is not assignable to parameter of type 'never'.
- Cause: By default, `useNavigation` is typed as `useNavigation<never>`
- TypeScript cannot infer the valid route names in navigation stack

Solution: Define a typed navigation stack and use it with `useNavigation`

Navigation Stack Type

Define a type for navigation stack to list all valid routes and their parameters

types.ts

```
1 export type RootStackParamList = {  
2   Home: undefined; // No parameters for Home screen  
3   Details: undefined; // No parameters for Details screen  
4   // Example with parameters: Profile: { id: string };  
5 };
```

- `undefined`: Indicates a screen expects no parameters
- `navigation.navigate("Details")`
- Parameterized routes: Define parameters as objects
- `navigation.navigate("Profile", { id: string })`

Typed Navigation Prop

Define the navigation prop type with `RootStackParamList`

types.ts

```
1 import { NativeStackNavigationProp } from "@react-navigation/native-stack"
2
3 export type RootStackParamList = {
4   Home: undefined;
5   Details: undefined;
6 };
7
8 export type NavigationProp = NativeStackNavigationProp<RootStackParamList
```

- `NavigationProp` ensures `navigate` accepts only valid route names with valid parameters

Typed Navigation Prop

Use `NavigationProp` with `useNavigation`

```
import { NavigationProp } from "../types";  
  
const navigation = useNavigation<NavigationProp>();
```

Typed Navigation Prop

Update [HomeScreen](#) to use typed navigation prop

screens/HomeScreen.tsx

```
1 import { View, Text, Button } from "react-native";
2 import { useNavigation } from "@react-navigation/native";
3 import { globalStyles } from "../styles/globalStyles";
4 import { NavigationProp } from "../types";
5
6 export default function HomeScreen() {
7   const navigation = useNavigation<NavigationProp>();
8
9   return (
10     <View style={globalStyles.container}>
11       <Text style={globalStyles.headerText}>Home Screen</Text>
12       <Button
13         title="Go to Details"
14         onPress={() => navigation.navigate("Details")}
15       />
16     </View>
17   );
18 }
```

Live Demo

screens/HomeScreen.tsx

```
1 import { View, Text } from "react-native";
2 import { useNavigation } from "@react-navigation/native";
3 import { globalStyles } from "../styles/globalStyles";
4 import { NavigationProp } from "../types";
5 import PrimaryButton from "../components/PrimaryButton";
6
7 export default function HomeScreen() {
8   const navigation = useNavigation<NavigationProp>();
9
10  return (
11    <View style={globalStyles.container}>
12      <Text style={globalStyles.headerText}>Home Screen</Text>
13      <PrimaryButton onPress={() => navigation.navigate("Details")}>
14        Go to Details
15      </PrimaryButton>
16    </View>
17  );
18 }
```

Update App.tsx

Ensure the navigation stack is typed in app's entry point

App.tsx

```
1 import { createStaticNavigation } from "@react-navigation/native";
2 import { createNativeStackNavigator } from "@react-navigation/native-stac
3 import HomeScreen from "../screens/HomeScreen";
4 import DetailsScreen from "../screens/DetailsScreen";
5 import { colors } from "../constants/colors";
6 import { RootStackParamList } from "../types";
7
8 const RootStack = createNativeStackNavigator<RootStackParamList>({
9   initialRouteName: "Home",
10  screenOptions: {
11    headerStyle: { backgroundColor: colors.primary },
12    headerTintColor: colors.white,
13  },
14  screens: {
15    Home: {
16      screen: HomeScreen,
17      options: {
18        title: "Overview",
19      },
```

Update DetailsScreen

screens/DetailsScreen.tsx

```
1 import { View, Text } from "react-native";
2 import { useNavigation } from "@react-navigation/native";
3 import { globalStyles } from "../styles/globalStyles";
4 import { NavigationProp } from "../types";
5 import PrimaryButton from "../components/PrimaryButton";
6
7 export default function DetailsScreen() {
8   const navigation = useNavigation<NavigationProp>();
9
10  return (
11    <View style={globalStyles.container}>
12      <Text style={globalStyles.headerText}>Details Screen</Text>
13      <PrimaryButton onPress={() => navigation.navigate("Home")}>
14        Go to Home
15      </PrimaryButton>
16    </View>
17  );
18 }
```

Header Layout

Inconsistent header layout across iOS and Android
`createNativeStackNavigator` applies platform-specific defaults for the header layout to mimic native behavior

- iOS: Centered title, back button on the left
- Android: Left-aligned title next to back arrow

Consistent Header

Standardize header layout across platforms

```
screenOptions: {  
  headerStyle: { backgroundColor: colors.primary },  
  headerTintColor: colors.white,  
  headerTitleAlign: "center", // Center titles on iOS and Android  
  headerBackTitle: "Back", // Standardize back button label  
},
```

Documentation: **Native Stack Navigator Options**

Recap: Navigation Basics

React Navigation: Component-Based navigation library for React Native and Web apps

Native Stack Navigator

- Static Configuration: `createNativeStackNavigator`, `createStaticNavigation`
- Screen Navigation: `useNavigation`, `navigate`
- Type-Safe Navigation: Typed navigation stack (`RootStackParamList`), Typed navigation prop (`NativeStackNavigationProp`)
- Custom headers for consistency across platforms

Dynamic Configuration

- `createNativeStackNavigator`: Returns an object with `Navigator` and `Screen` components
- `Navigator`: Defines the navigation structure
- `Screen`: Maps a route name to a component

```
const Stack = createNativeStackNavigator();  
function RootStack() {  
  return (  
    <Stack.Navigator>  
      <Stack.Screen name="Home" component={HomeScreen} />  
    </Stack.Navigator>  
  );  
}
```

React components enable runtime changes via state/props

Dynamic Configuration

- `createNativeStackNavigator`
- `NavigationContainer`: Manages navigation state and history
- Must wrap all the navigators in the app
- Typically rendered at app's root (`App.tsx`)

App.tsx

```
export default function App() {  
  return (  
    <NavigationContainer>  
      <RootStack />  
    </NavigationContainer>  
  );  
}
```

Example

Define a typed stack navigator with two screens

App.tsx

```
1 import { NavigationContainer } from "@react-navigation/native";
2 import { createNativeStackNavigator } from "@react-navigation/native-stac
3 import HomeScreen from "../screens/HomeScreen";
4 import DetailsScreen from "../screens/DetailsScreen";
5 import { colors } from "../constants/colors";
6 import { RootStackParamList } from "../types";
7
8 const Stack = createNativeStackNavigator<RootStackParamList>();
9
10 function RootStack() {
11   return (
12     <Stack.Navigator>
13       <Stack.Screen name="Home" component={HomeScreen} />
14       <Stack.Screen name="Details" component={DetailsScreen} />
15     </Stack.Navigator>
16   );
17 }
18
19 export default function App() {
```

Screen Options

App.tsx

```
1 function RootStack() {
2   return (
3     <Stack.Navigator
4       initialRouteName="Home"
5       screenOptions={{
6         headerStyle: { backgroundColor: colors.primary },
7         headerTintColor: colors.white,
8         headerTitleAlign: "center",
9         headerBackTitle: "Back",
10      }}
11   >
12     <Stack.Screen
13       name="Home"
14       component={HomeScreen}
15       options={{ title: "Overview" }}
16     />
17     <Stack.Screen name="Details" component={DetailsScreen} />
18   </Stack.Navigator>
19 );
```

Pass Props to Screens

React Navigation provides default props to screens

- `route`: Information about the current route
- `route.name`: Name of the current route (e.g., “Details”)
- `route.params`: Any data passed when navigating to this screen

```
function DetailsScreen({ route }) {  
  return <Text>Received ID: {route.params.id}</Text>;  
}
```

Example

Pass ID from Home screen to Details screen

- **HomeScreen**: User inputs an id and navigates to **DetailsScreen**
- **DetailsScreen**: Displays the received id

Change type definition

types.ts

```
export type RootStackParamList = {  
  Home: undefined;  
  Details: { id: string };  
};
```

Next Step: Pass `route.params.id` from **HomeScreen** to **DetailsScreen**

Lecture Summary

- **Styling and UI Design in React Native**
- Responsive design with `Dimensions.get` and `useWindowDimensions`
- Dark/Light mode support using `Appearance` and `useColorScheme`
- Centralized styles and colors for consistency
- Advanced styling tools: Animations, Accessibility, UI libraries

Lecture Summary

- **Navigation with React Navigation**
 - Core concepts: Screens, Navigators, Routes
 - Native Stack Navigator: Static configuration
 - Type-safe navigation with `RootStackParamList` + `NavigationProp`, `useNavigation`
 - Consistent header styling across platforms

Speaker notes